Exercise – 7 → Financial Forecasting

What is Recursion?

Recursion is a programming technique in which a method calls itself to solve a problem by breaking it into smaller subproblems.

A recursive function consists of:

- **Base Case**: Condition that stops recursion.
- **Recursive Case**: Function calls itself with a smaller input.

For financial forecasting, assume:

- Current Value = Initial investment/value
- Growth Rate = Annual percentage growth
- Years = Number of years to forecast

Formula:

Future Value = Present Value × (1 + Growth Rate)

```
[Running] cd "c:\Users\savit\OneDrive\Des
Forecasting\" && javac FinancialForecast.
Predicted value after 5 years: 16105.10
```

Time Complexity

The recursive function executes once for each year.

For n years:

T(n) = T(n-1) + O(1)

Therefore: **Time Complexity = O(n)**

Space Complexity

Each recursive call occupies stack memory.

Space Complexity = O(n)

Optimization

For large values of years, recursion creates many function calls, increasing memory usage and risking stack overflow.

Solution: Mathematical Formula
Use compound growth directly:
double futureValue = currentValue * Math.pow(1 + growthRate, years);

Time Complexity:O(1)

This is the most efficient approach for financial forecasting.